Unit 2: DATA COLLECTION, CLEANING, AND PREPROCES

Data Collection Methods: Data Sources: Relational Databases, APIs, Web Scraping,

Data Collection Methods:

Data Collection refers to the systematic process of gathering, measuring, and analyzing information from various sources to get a complete and accurate picture of an area of interest. Data collection is a critical step in any research or data-driven decision-making process, ensuring the accuracy and reliability of the results obtained. By employing various methods of data collection researchers and organizations can gather the necessary data to support their objectives effectively.

What is Data Collection?

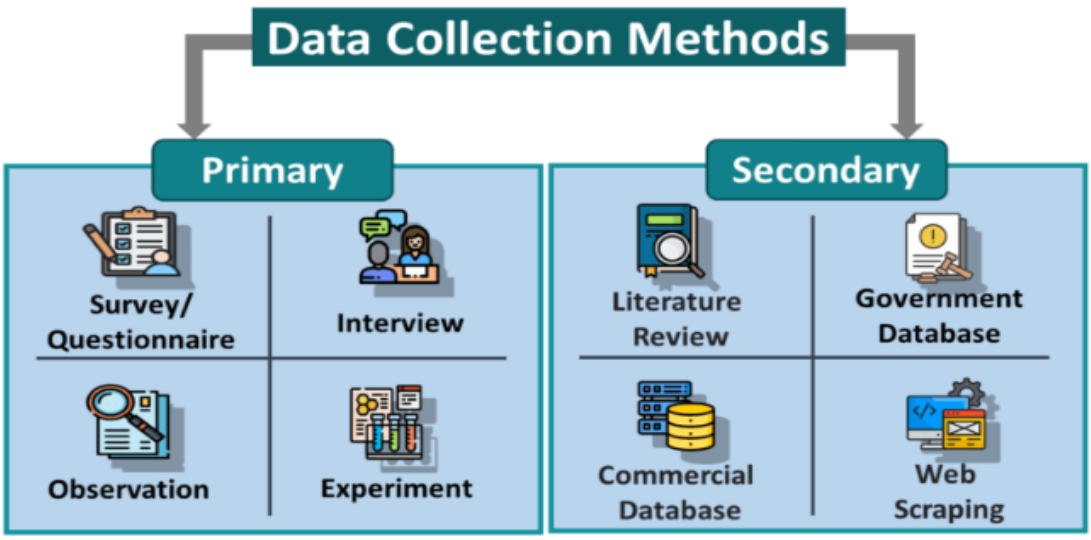
Data Collection is the process of collecting information from relevant sources to find a solution to the given statistical inquiry. Collection of Data is the first and foremost step in a statistical investigation. It's an essential step because it helps us make informed decisions, spot trends, and measure progress.

Different methods of collecting data include

- Interviews
- Questionnaires
- Observations
- Experiments

Published Sources and Unpublished Sources

Here, statistical inquiry means an investigation by any agency on a topic in which the investigator collects the relevant quantitative information. In simple terms, a statistical inquiry is a search for truth by using statistical methods of collection, compiling, analysis, interpretation, etc. The basic problem for any statistical inquiry is the collection of facts and figures related to this specific phenomenon that is being studied. Therefore, the basic purpose of data collection is collecting evidence to reach a sound and clear solution to a problem.



Primary Data

Primary data refers to information collected directly from first-hand sources specifically for a particular research purpose. This type of data is gathered through various methods, including surveys, interviews, experiments, observations, and focus groups. One of the main advantages of primary data is that it provides current, relevant, and specific information tailored to the researcher's needs, offering a high level of accuracy and control over data quality.

Methods of Collecting Primary Data

There are a number of methods of collecting primary data, Some of the common methods are as follows:

1. Interviews: Collect data through direct, one-to-one conversations with individuals. The investigator asks questions either directly from the source or from its indirect links.

1.Direct Personal Investigation: The method of direct personal investigation involves collecting data personally from the source of origin. In simple words, the investigator makes direct contact with the person from whom he/she wants to obtain information.

For example, direct contact with the household women to obtain information about their daily routine and schedule.

2.Indirect Oral Investigation: In the indirect oral investigation method of collecting primary data, the investigator does not make direct contact with the person from whom he/she needs information, instead they collect the data orally from some other person who has the necessary required information.

For example, collecting data of employees from their superiors or managers.

•Advantage: Provides real-time, natural data; no reliance on self-reported information.

•Disadvantage: Observer bias; limited to what can be seen; may influence subjects' behavior.

•Suitable Use Case: Behavioral studies, user experience research.

2. Questionnaires: Collect data by asking people a set of questions, either online, on paper, or face-to-face. In this method the investigator prepares a questionnaire to collect Information through Questionnaires and Schedules , while keeping in mind the motive of the study, . The investigator can collect data through the questionnaire in two ways:

1.Mailing Method: This method involves mailing the questionnaires to the informants for the collection of data. The investigator attaches a letter with the questionnaire in the mail to define the purpose of the study or research.

2.Enumerator's Method: This method involves the preparation of a questionnaire according to the purpose of the study or research. However, in this case, the enumerator reaches out to the informants himself with the prepared questionnaire.

•Advantage: Can reach a large audience quickly and cost-effectively.

•Disadvantage: Responses may be biased or inaccurate; low response rates.

•Suitable Use Case: Customer satisfaction surveys, market research.

3. Observations: The observation method involves collecting data by watching and recording behaviors, events, or conditions as they naturally occur. The observer systematically watches and notes specific aspects of a subject's behavior or the environment, either covertly or overtly.

•Advantage: Provides real-time, authentic data without reliance on self-reported information.

•Disadvantage: Observer bias can influence the results, and the presence of an observer might alter subjects' behavior.

•Suitable Use Case: Studying user interactions with a product in a natural setting, monitoring wildlife behavior, or assessing classroom dynamics.

4. Experiments: The experiment method involves manipulating one or more variables to determine their effect on another variable, within a controlled environment. Researchers create two groups (control and experimental), apply the treatment or variable to the experimental group, and compare the outcomes between the groups.

•Advantage: Allows for the establishment of cause-and-effect relationships with high precision.

•Disadvantage: Experiments can be artificial, limiting the ability to generalize findings to real-world settings, and they can be resource-intensive.

•Suitable Use Case: Testing the efficacy of a new drug, assessing the impact of a new teaching method, or evaluating the effect of a marketing campaign.

5. Focus Group: The focus group method involves gathering a small group of people to discuss a specific topic or product, facilitated by a moderator. A group of 6-12 participants engages in a guided discussion led by a moderator who asks open-ended questions to elicit opinions, attitudes, and perceptions.

•Advantage: Provides in-depth insights and diverse perspectives through interactive discussions, revealing the reasoning behind participants' thoughts and feelings.

•Disadvantage: Results can be influenced by dominant participants or groupthink, and the findings are not easily generalizable due to the small, non-representative sample size.

•Suitable Use Case: Exploring customer attitudes towards a new product, gathering feedback on a marketing campaign, or understanding public opinion on social issues.

6. Information from Local Sources or Correspondents: In this method, for the collection of data, the investigator appoints correspondents or local persons at various places, which are then furnished by them to the investigator. With the help of correspondents and local persons, the investigators can cover a wide area.

Secondary Data

Secondary data refers to information that has already been collected, processed, and published by others. This type of data can be sourced from existing research papers, government reports, books, statistical databases, and company records. The advantage of secondary data is that it is readily available and often free or less expensive to obtain compared to primary data. It saves time and resources since the data collection phase has already been completed.

Methods of Collecting Secondary Data

Secondary data can be collected through different published and unpublished sources. Some of them are as follows:

1. Published Sources

Government Publications: Government publishes different documents which consists of different varieties of information or data published by the Ministries, Central and State Governments in India as their routine activity. As the government publishes these Statistics, they are fairly reliable to the investigator. Examples of Government publications on Statistics are the Annual Survey of Industries, Statistical Abstract of India, etc.

Semi-Government Publications: Different Semi-Government bodies also publish data related to health, education, deaths and births. These kinds of data are also reliable and used by different informants. Some examples of semi-government bodies are Metropolitan Councils, Municipalities, etc.

Publications of Trade Associations: Various big trade associations collect and publish data from their research and statistical divisions of different trading activities and their aspects. For example, data published by Sugar Mills Association regarding different sugar mills in India.

Journals and Papers: Different newspapers and magazines provide a variety of statistical data in their writings, which are used by different investigators for their studies.

International Publications: Different international organizations like IMF, UNO, ILO, World Bank, etc., publish a variety of statistical information which are used as secondary data.

Publications of Research Institutions: Research institutions and universities also publish their research activities and their findings, which are used by different investigators as secondary data. For example National Council of Applied Economics, the Indian Statistical Institute, etc.

2. Unpublished Sources

Unpublished sources are another source of collecting secondary data. The data in unpublished sources is collected by different government organizations and other organizations. These organizations usually collect data for their self-use and are not published anywhere. For example, research work done by professors, professionals, teachers and records maintained by business and private enterprises.

Relational Databases

Description

Relational databases store data in structured formats using tables, where each table consists of rows and columns. Data is organized into relationships, allowing for efficient querying and management.

Key Features

• Structured Data: Data is organized in a predefined schema, making it easier to enforce data integrity.

• SQL (Structured Query Language): A standard language used for querying and managing relational databases.

• ACID Properties: Ensures reliable transactions through Atomicity, Consistency, Isolation, and Durability.

Advantages

• Data Integrity: Relationships between tables help maintain data consistency.

• Complex Queries: SQL allows for sophisticated queries to retrieve specific data sets.

• Scalability: Can handle large amounts of data effectively.

Common Use Cases

• Business Applications: Customer relationship management (CRM) systems, inventory management.

• Financial Data Management: Transaction processing systems, accounting databases.

APIs (Application Programming Interfaces)

Description

APIs are sets of protocols and tools that allow different software applications to communicate with each other. They enable access to data or services from external systems.

Key Features

• RESTful and SOAP Protocols: Common architectures for APIs, with REST being more widely used for web services.

• Data Formats: Typically return data in formats like JSON or XML, which are easy to parse.

Advantages

• Real-time Data Access: Allows for the retrieval of up-to-date information from external sources.

• Ease of Integration: Facilitates the incorporation of external data into applications without significant overhead.

• Scalability: Can handle numerous requests simultaneously.

Common Use Cases

• Social Media Data: Accessing user posts, comments, and interactions from platforms like Twitter or Facebook.

• Financial Data: Retrieving stock prices, exchange rates, and economic indicators from financial APIs.

Web Scraping

Description

Web scraping involves extracting data from websites using automated tools or scripts. It can be used to collect information that is not readily available through APIs or other means.

Key Features

• HTML Parsing: Extracts data by parsing the HTML structure of web pages.

• Automation: Scripts can be programmed to run at specified intervals to gather data.

Advantages

• Access to Unstructured Data: Can collect data from any publicly available webpage, including text, images, and tables.

• Cost-effective: Often less expensive than purchasing datasets from data providers.

• Flexibility: Can target specific web pages and elements for tailored data collection.

Common Use Cases

• Market Research: Gathering pricing information and product details from e-commerce sites.

• Sentiment Analysis: Collecting user reviews and comments from various platforms to analyze public opinion.

Tools for Data Collection: BeautifulSoup, Requests, Scrapy,

1. BeautifulSoup

Description

BeautifulSoup is a Python library used for parsing HTML and XML documents. It creates parse trees from page source code, making it easy to extract data from web pages.

Beautiful Soup helps us do various tasks like parsing, data extraction, data manipulation, etc.

Key Features

• Easy Navigation: Allows for easy traversal of the parse tree using simple methods.

• Flexible Parsing: Can parse poorly structured HTML, making it robust for real-world web pages.

• Integration: Works well with other libraries, especially Requests.

Advantages

• User-Friendly: Simple API makes it accessible for beginners.

• Powerful Searching: Supports searching for elements using tags, classes, and IDs.

Common Use Cases

• Extracting product details from e-commerce websites.

• Scraping articles or blog posts for text analysis.

Example Code

```
import BeautifulSoup

import requests

# Fetch the webpage

url = 'http://example.com'

response = requests.get(url)


# Parse the HTML

soup = BeautifulSoup(response.content, 'html.parser')


# Extract specific data

titles = soup.find_all('h2')

for title in titles:

    print(title.text)
```

2. Requests

Description

Requests is a popular Python library for making HTTP requests. It simplifies the process of sending HTTP requests and handling responses.

Key Features

• User-Friendly: Simplifies the syntax for making requests compared to the built-in urllib.

• Support for Sessions: Allows persistent connections to maintain session data.

• Easy Handling of JSON: Built-in methods for parsing JSON responses.

Advantages

• HTTP Methods: Supports various methods like GET, POST, PUT, DELETE, etc.

• Headers and Parameters: Easily customize requests with headers and query parameters.

Common Use Cases

• Accessing APIs to fetch data.

• Downloading web pages for further scraping.

Example Code

```
import requests

# Make a GET request

url = 'http://example.com/api/data'

response = requests.get(url)


# Check if the request was successful

if response.status_code == 200:

    data = response.json()  # Parse JSON response

    print(data)

else:

    print("Failed to retrieve data")
```

3. Scrapy

Description

Scrapy is a powerful and popular web scraping framework for Python. It provides a comprehensive set of tools for crawling websites and extracting structured data.

Key Features

• Built-In Scraping Mechanism: Comes with tools to manage requests, follow links, and extract data.

• Asynchronous Processing: Utilizes Twisted for handling multiple requests concurrently, making it fast.

• Data Storage: Supports exporting scraped data in various formats like JSON, CSV, and XML.

Advantages

• Robustness: Handles large-scale scraping tasks effectively.

• Spider Management: Easily manage multiple spiders (scraping scripts) for different websites.

• Middleware Support: Allows customization of the request and response processing pipeline.

Common Use Cases

• Crawling entire websites to build datasets.

• Extracting data from complex sites with pagination and login requirements.

Example Code

This is a simplified example. In practice, you'd run this through Scrapy's command-line interface.

```
import scrapy

class MySpider(scrapy.Spider):

    name = 'my_spider'

    start_urls = ['http://example.com']


    def parse(self, response):

        titles = response.css('h2::text').getall()

        for title in titles:

            yield {'title': title}
```

To run this spider, you'd typically save it in a file and run it via the Scrapy command line.

Data Cleaning and Preprocessing : Handling Missing Data: Imputation, Deletion

Data cleaning and preprocessing are critical steps in data science, particularly when dealing with missing data. Here's a detailed overview of two common methods for handling missing data: Imputation and Deletion.

Handling Missing Data

1. Imputation

Imputation involves filling in the missing values with estimated values based on other available information. This method helps retain the dataset's size and can reduce bias in analysis.

Common Imputation Techniques

• Mean/Median/Mode Imputation:

- Mean: Replace missing values with the mean of the column. Useful for normally distributed data.
- Median: Use the median when the data is skewed, as it is less affected by outliers.
- Mode: For categorical data, replace missing values with the most frequent category.

```
import pandas as pd

# Sample DataFrame

df = pd.DataFrame({'A': [1, 2, None, 4], 'B': ['cat', None, 'dog', 'cat']})


# Mean imputation for numeric column

df['A'].fillna(df['A'].mean(), inplace=True)


# Mode imputation for categorical column

df['B'].fillna(df['B'].mode()[0], inplace=True)
```

• K-Nearest Neighbors (KNN) Imputation:

- Uses the feature values from similar observations (nearest neighbors) to estimate missing values.

• Regression Imputation:

- Create a regression model to predict the missing values based on other variables in the dataset.

• Interpolation:

- Useful for time series data, where missing values are filled in based on the values before and after them.

Advantages of Imputation

• Maintains the dataset's size, which is especially important for small datasets.

• Helps avoid the potential bias that may arise from removing data points.

Disadvantages of Imputation

• Can introduce noise if the imputation model is not accurate.

• May lead to overfitting if complex models are used for imputation.

2. Deletion

Deletion involves removing data points with missing values from the dataset. This method is straightforward but can result in loss of information.

Types of Deletion

• Listwise Deletion (Complete Case Analysis):

- Remove any row that has at least one missing value. This is simple and preserves the integrity of the remaining data.

Listwise deletion

df_cleaned = df.dropna()

• Pairwise Deletion:

- Only exclude missing values when they affect a specific analysis or calculation. This approach allows more data to be retained for correlation or regression analyses.

Advantages of Deletion

• Simple and straightforward to implement.

• Maintains the integrity of the remaining data without introducing bias from imputation.

Disadvantages of Deletion

• Can lead to significant data loss, particularly in small datasets.

• May introduce bias if the missing data is not randomly distributed (i.e., not missing at random).

Data Transformation: Normalization, Standardization, Encoding

Data normalization is a technique used in data mining to transform the values of a dataset into a common scale. This is important because many machine learning algorithms are sensitive to the scale of the input features and can produce better results when the data is normalized.

There are several different normalization techniques that can be used in data mining, including:

Methods of Data Normalization –

1.Min-Max normalization: This technique scales the values of a feature to a range between 0 and 1. This is done by subtracting the minimum value of the feature from each value, and then dividing by the range of the feature.

$$X' = \frac{X - Xmin}{Xmax - Xmin}$$

Example:

Suppose that the minimum and maximum values for the attribute *income* are $12,000 and $98,000, respectively. We would like to map *income* to the range [0.0, 1.0]. By min-max normalization, a value of $73,600 for *income* is transformed

$$\frac{73{,}600 - 12{,}000}{98{,}000 - 12{,}000}(1.0 - 0) + 0 = 0.716.$$

2.Z-score normalization: Z-score normalization refers to the process of normalizing every value in a dataset such that the mean of all of the values is 0 and the standard deviation is 1.

We use the following formula to perform a z-score normalization on every value in a dataset:

New value = (x – μ) / σ

where:

- **x**: Original value
- **μ**: Mean of data
- **σ**: Standard deviation of data

The following example shows how to perform z-score normalization on a dataset in practice.

Example: Performing Z-Score Normalization

Suppose we have the following dataset:

Data	3	5	5	8	9	12	12	13	15	16	17	19	22	24	25	134

Using a calculator, we can find that the mean of the dataset is **21.2** and the standard deviation is **29.8**.

To perform a z-score normalization on the first value in the dataset, we can use the following formula:

- New value = (x – μ) / σ
- New value = (3 – 21.2) / 29.8
- New value = -0.61

We can use this formula to perform a z-score normalization n every value in the dataset:

Data	Z-Score Normalized Value
3	-0.61
5	-0.54
5	-0.54
8	-0.44
9	-0.41
12	-0.31
12	-0.31
13	-0.28
15	-0.21
16	-0.17
17	-0.14
19	-0.07
22	0.03
24	0.09
25	0.13
134	3.79

3.Decimal Scaling Method For Normalization –

It normalizes by moving the decimal point of values of the data. To normalize the data by this technique, we divide each value of the data by the maximum absolute value of data. The data value, vi, of data is normalized to vi‘ by using the formula below –

where j is the smallest integer such that max(|vi‘|)<1.

Example –

Let the input data is: -10, 201, 301, -401, 501, 601, 701

To normalize the above data,

Step 1: Maximum absolute value in given data(m): 701

Step 2: Divide the given data by 1000 (i.e j=3)

Result: The normalized data is: -0.01, 0.201, 0.301, -0.401, 0.501, 0.601, 0.701

4. Logarithmic transformation: This technique applies a logarithmic transformation to the values of a feature. This can be useful for data with a wide range of values, as it can help to reduce the impact of outliers.

5. Root transformation: This technique applies a square root transformation to the values of a feature. This can be useful for data with a wide range of values, as it can help to reduce the impact of outliers.

6. It's important to note that normalization should be applied only to the input features, not the target variable, and that different normalization technique may work better for different types of data and models.

In conclusion, normalization is an important step in data mining, as it can help to improve the performance of machine learning algorithms by scaling the input features to a common scale. This can help to reduce the impact of outliers and improve the accuracy of the model.

Normalization is used to scale the data of an attribute so that it falls in a smaller range, such as -1.0 to 1.0 or 0.0 to 1.0. It is generally useful for classification algorithms.

Need of Normalization –

Normalization is generally required when we are dealing with attributes on a different scale, otherwise, it may lead to a dilution in effectiveness of an important equally important attribute(on lower scale) because of other attribute having values on larger scale. In simple words, when multiple attributes are there but attributes have values on different scales, this may lead to poor data models while performing data mining operations. So they are normalized to bring all the attributes on the same scale.

ADVANTAGES OR DISADVANTAGES:

Data normalization in data mining can have a number of advantages and disadvantages.

Advantages:

1. Improved performance of machine learning algorithms: Normalization can help to improve the performance of machine learning algorithms by scaling the input features to a common scale. This can help to reduce the impact of outliers and improve the accuracy of the model.

2. Better handling of outliers: Normalization can help to reduce the impact of outliers by scaling the data to a common scale, which can make the outliers less influential.

3. Improved interpretability of results: Normalization can make it easier to interpret the results of a machine learning model, as the inputs will be on a common scale.

4. Better generalization: Normalization can help to improve the generalization of a model, by reducing the impact of outliers and by making the model less sensitive to the scale of the inputs.

Disadvantages:

1. Loss of information: Normalization can result in a loss of information if the original scale of the input features is important.

2. Impact on outliers: Normalization can make it harder to detect outliers as they will be scaled along with the rest of the data.

3. Impact on interpretability: Normalization can make it harder to interpret the results of a machine learning model, as the inputs will be on a common scale, which may not align with the original scale of the data.

4. Additional computational costs: Normalization can add additional computational costs to the data mining process, as it requires additional processing time to scale the data.

5. In conclusion, data normalization can have both advantages and disadvantages. It can improve the performance of machine learning algorithms and make it easier to interpret the results. However, it can also result in a loss of information and make it harder to detect outliers. It's important to weigh the pros and cons of data normalization and carefully assess the risks and benefits before implementing it.

Example

Let's say the exam is conducted in two shifts shift A and shift B and questions in shift A were relatively easy as compared to that of shift B. Because questions were relatively easy in shift A maximum marks scored by a candidate out of 300 in shift A is 280 and minimum is 80 on the other hand maximum and minimum marks scored by a candidate in shift B is 250 and 50 respectively.

So we cannot compare the score of a candidate who scored 150 in shift A to a candidate who scored the same in shift B.

Hence we normalize the scores

The normalized score of a candidate who scored 150 in shift A will be calculated as follows

For simplicity sake let's name it Xa

Xa =150-Xmin/(Xmin-Xmax)

Xmax = 280

Xmin = 80

Putting these values we get

Xa= 150–80/(280–80)

Xa =0.35

The normalized score of a candidate who scored 150 in shift B will be

For simplicity sake let’s name it Xb

Xb=150-Xmin/(Xmin-Xmax)

Xmax = 250

Xmin = 50

Putting these values we get

Xb = 150–50/(250–50)

Xb =0.5

Here we can see Xb>Xa

Total marks in CAT exam=300

If you want to compare out of 300 then

Xa *300 =105

Xb *300 = 150

I think now it will be clearer to understand what normalization is and why we need it.

All right, let's jump to the second one

Standardization

Standardization is another scaling method where the values are centered around mean with a unit standard deviation. It means if we will calculate mean and standard deviation of standard scores it will be 0 and 1 respectively.

The formula for standardized values:

$$z = \frac{x - \mu}{\sigma}$$

Where,

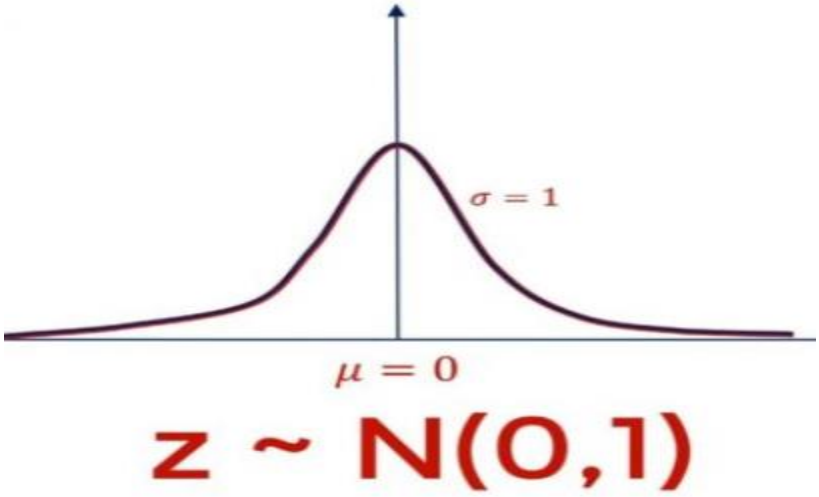
μ= mean of the given distribution

σ = standard deviation of the given distribution

This Z is called standard score and it represents the number of standard deviations above or below the mean that a specific observation falls.

i.e. If Z=2, it means that the observation lies two standard deviations above the mean.

If we plot these standard scores it will be a normal distribution with mean at 0 and the standard deviation equals to 1.



The normal distribution with mean = 0 and σ =1 is also known as standard normal distribution and is denoted by N(0,1).

Getting too technical, isn't it?

Let’s solve an **example** for better understanding.

Let’s assume you and your friend study in different universities where the grading system is different. You get your score of 85 in a test. The mean grade of the class is 75 and a standard deviation of 5. Your friend got a grade of 615 and the mean grade of the class is 600 with a standard deviation of 50. How you are going to say who is performing better? As grade 85 can’t be compared to 615.

Here comes the role of standardization as it allows us to compare the scores with different metrics directly and make a statement about them.

Your Z score = 85–75/5 = 2

It means you are 2 standard deviations above the average grade.

Your friend’s Z score = 615–600/50 = 0.3

It means you are 0.3 standard deviations above the average grade.

Difference between Normalization and Standardization

S.NO.	Normalization	Standardization
1.	Minimum and maximum value of features are used for scaling	Mean and standard deviation is used for scaling.
2.	It is used when features are of different scales.	It is used when we want to ensure zero mean and unit standard deviation.
3.	Scales values between [0, 1] or [-1, 1].	It is not bounded to a certain range.
4.	It is really affected by outliers.	It is much less affected by outliers.
5.	Scikit-Learn provides a transformer called MinMaxScaler for Normalization.	Scikit-Learn provides a transformer called StandardScaler for standardization.
6.	This transformation squishes the n-	It translates the data to the mean

S.NO.	Normalization	Standardization
	dimensional data into an n-dimensional unit hypercube.	vector of original data to the origin and squishes or expands.
7.	It is useful when we don’t know about the distribution	It is useful when the feature distribution is Normal or Gaussian.
8.	It is a often called as Scaling Normalization	It is a often called as Z-Score Normalization.

Encoding

Data Encoding is an important pre-processing step in Machine Learning. It refers to the process of converting categorical or textual data into numerical format, so that it can be used as input for algorithms to process. The reason for encoding is that most machine learning algorithms work with numbers and not with text or categorical variables.

The Main focus of this Notebook is to understand

- **What is Categorical Data and Why to Encode Data**
- **Different Data Encoding Techniques**
- **How to Implement it**.

What is Categorical Data?

When we collect data, we often encounter different types of variables. One such type is categorical variables. Categorical variables are usually represented as ‘strings’ or ‘categories’ and are finite in number.

There are two types of categorical data -

• ***Ordinal Data***

• ***Nominal Data***

Here are a few examples of categorical variables:

• **Places**: Delhi, Mumbai, Ahmedabad, Bangalore, etc.

• **Departments**: Finance, Human resources, IT, Production.

• **Grades**: A, A-, B+, B, B- etc.

Ordinal Data:

The categories of ordinal data have an **Inherent Order**. This means that the categories can be **Ranked** or ordered from highest to lowest or vice versa.

For example, the variable "highest degree a person has" is an ordinal variable. The categories (High school, Diploma, Bachelors, Masters, PhD) can be ranked in order of the level of education attained.

Nominal Data:

The categories of nominal data **do not have an Inherent Order**. This means that the categories cannot be ranked or ordered.

For example, the variable "city where a person lives" is a nominal variable. The categories (Delhi, Mumbai, Ahmedabad, Bangalore, etc.) cannot be ranked or ordered.

What is Data Encoding?

Data Encoding is an important pre-processing step in Machine Learning. It refers to the process of converting categorical or textual data into numerical format, so that it can be used as input for algorithms to process. The reason for encoding is that most machine learning algorithms work with numbers and not with text or categorical variables.

Why it is Important?

• Most machine learning algorithms work only with numerical data, so categorical variables (such as text labels) must be transformed into numerical values.

• This allows the model to identify patterns in the data and make predictions based on those patterns.

• Encoding also helps to prevent bias in the model by ensuring that all features are equally weighted.

• The choice of encoding method can have a significant impact on model performance, so it is important to choose an appropriate encoding technique based on the nature of the data and the specific requirements of the model.

There are several methods for encoding categorical variables, including

1. One-Hot Encoding

2. Dummy Encoding

3.Ordinal Encoding

4. Binary Encoding

5. Count Encoding

6. Target Encoding

Let’s take a closer look at each of these methods.

One-Hot Encoding:

• One-Hot Encoding is the **Most Common** method for encoding **Categorical** variables.

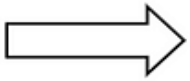
• a **Binary Column** is created for each **Unique Category** in the variable.

• If a category is present in a sample, the corresponding column is set to 1, and all other columns are set to 0.

• For example, if a variable has three categories ‘A’, ‘B’ and ‘C’, three columns will be created and a sample with category ‘B’ will have the value [0,1,0].

One-Hot Encoding

Places
New York
Boston
Chicago
California
New Jersey



New York	Boston	Chicago	California	New Jersey
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

```
# One-Hot Encoding:
# create a sample dataframe with a categorical variable
df = pd.DataFrame({'color': ['red', 'green', 'blue', 'red']})

# perform one-hot encoding on the 'color' column
one_hot = pd.get_dummies(df['color'])

# concatenate the one-hot encoding with the original dataframe
df1 = pd.concat([df, one_hot], axis=1)

# drop the original 'color' column
df1 = df1.drop('color', axis=1)
```

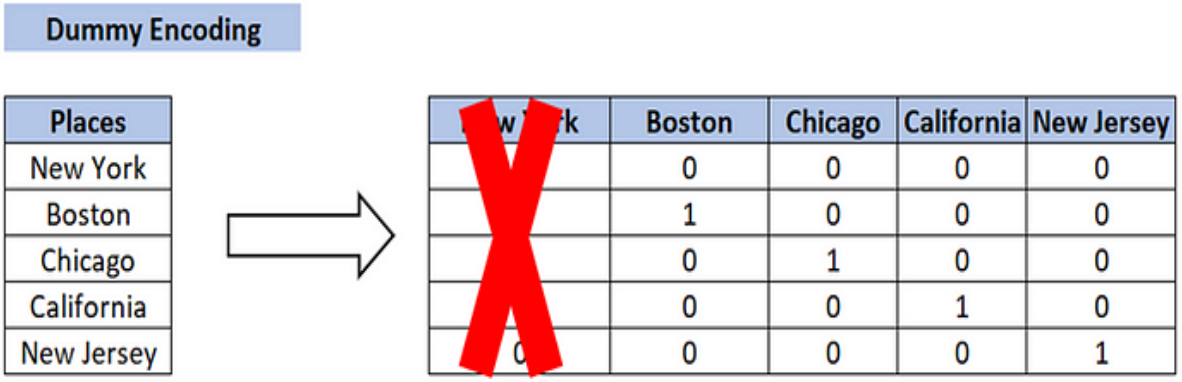
Dummy Encoding

• Dummy coding scheme is **similar to one-hot encoding**.

• This categorical data encoding method transforms the categorical variable into a set of binary variables [0/1].

• In the case of **one-hot encoding**, for N categories in a variable, it uses N binary variables.

• The dummy encoding is a small improvement over one-hot-encoding. Dummy encoding uses N-1 features to represent N labels/categories.



One-Hot Encoding vs Dummy Encoding:

One-Hot Encoding *— N categories in a variable,* ***N*** *binary variables.*

Dummy encoding *— N categories in a variable,* ***N-1*** *binary variables.*

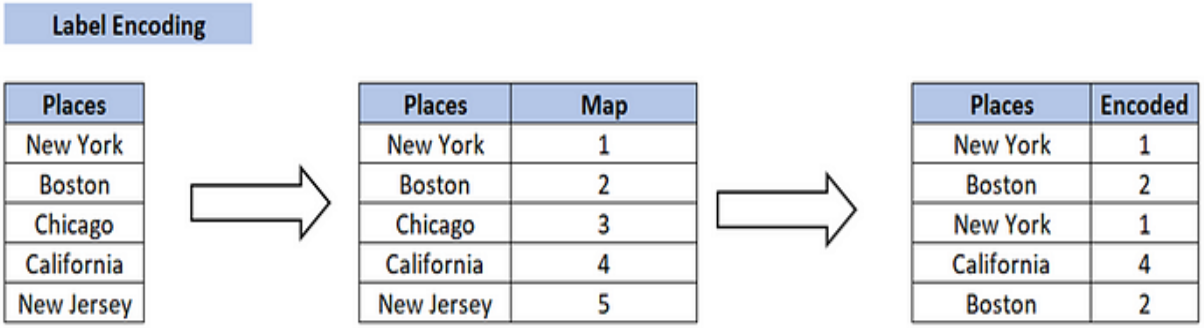
```
# Create a sample dataframe with categorical variable
data = {'Color': ['Red', 'Green', 'Blue', 'Red', 'Blue']}
df = pd.DataFrame(data)

# Use get_dummies() function for dummy encoding
dummy_df = pd.get_dummies(df['Color'], drop_first=True, prefix='Color')

# Concatenate the dummy dataframe with the original dataframe
df = pd.concat([df, dummy_df], axis=1)
```

Label Encoding:

- Each unique category is assigned a **Unique Integer** value.
- This is a simpler encoding method, but it has a **Drawback** in that the assigned integers may be **misinterpreted** by the machine learning algorithm **as having an Ordered Relationship** when in fact they **do not**.




```
from sklearn.preprocessing import LabelEncoder

# Create a sample dataframe with categorical data
df = pd.DataFrame({'color': ['red', 'green', 'blue', 'red', 'green']})

print(f"Before Encoding the Data:\n\n{df}\n")

# Create a LabelEncoder object
le = LabelEncoder()

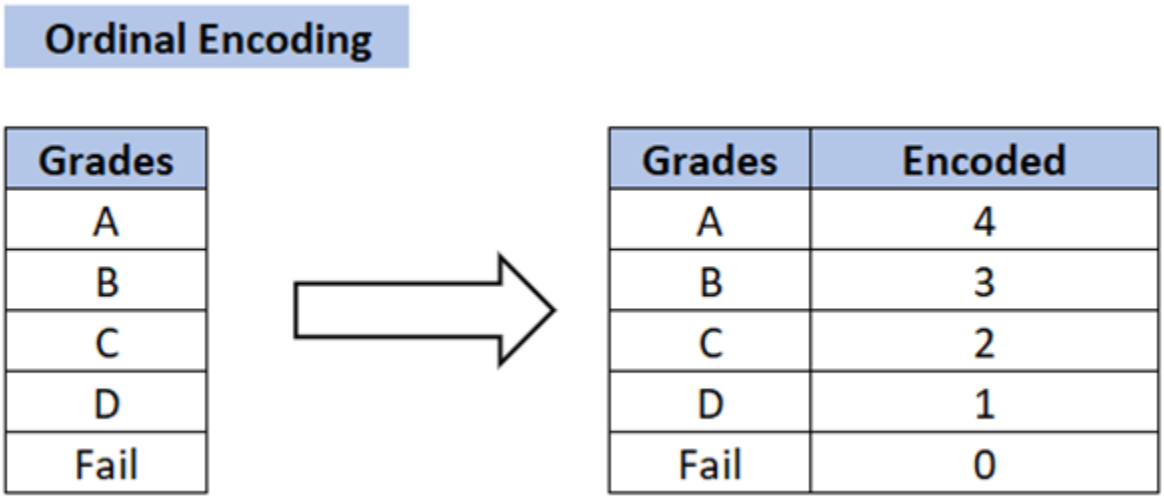
# Fit and transform the categorical data
df['color_label'] = le.fit_transform(df['color'])
```

Ordinal Encoding:

• Ordinal Encoding is used when the **categories** in a variable have a **Natural Ordering**.

• In this method, the **categories are assigned a numerical value** based on their order, such as 1, 2, 3, etc.

For example, if a variable has categories **'Low', 'Medium' and 'High'**, they can be assigned the values **1, 2, and 3**, respectively.



```
# Ordinal Encoding:
# create a sample dataframe with a categorical variable
df = pd.DataFrame({'quality': ['low', 'medium', 'high', 'medium']})
print(f"Before Encoding the Data:\n\n{df}\n")

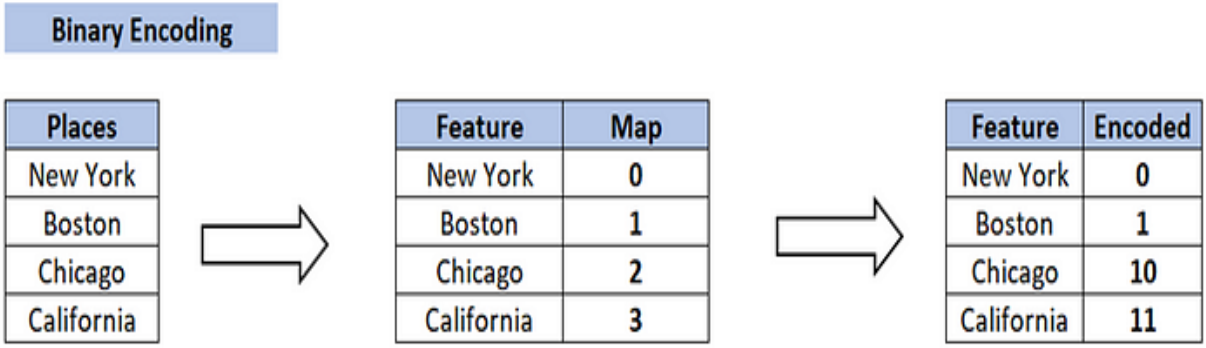
# specify the order of the categories
quality_map = {'low': 0, 'medium': 1, 'high': 2}

# perform ordinal encoding on the 'quality' column
df['quality_map'] = df['quality'].map(quality_map)
```

Binary Encoding:

• Binary Encoding is similar to One-Hot Encoding, but instead of creating a separate column for each category, the categories are represented as binary digits.

For example, if a variable has four categories ‘A’, ‘B’, ‘C’ and ‘D’, they can be represented as 0001, 0010, 0100 and 1000, respectively.




```
# Binary Encoding:

import pandas as pd

# create a sample dataframe with a categorical variable
df = pd.DataFrame({'animal': ['cat', 'dog', 'bird', 'cat']})
print(f"Before Encoding the Data:\n\n{df}\n")

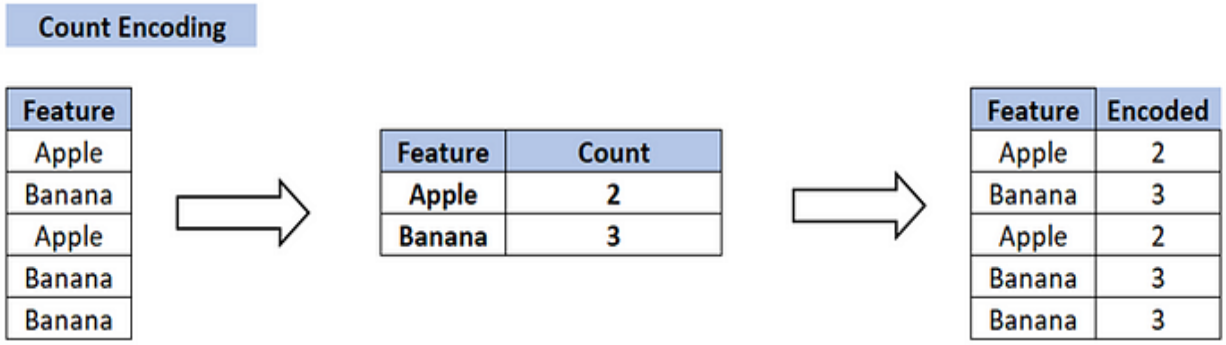
# perform binary encoding on the 'animal' column
animal_map = {'cat': 0, 'dog': 1, 'bird': 2}
df['animal'] = df['animal'].map(animal_map)
df['animal'] = df['animal'].apply(lambda x: format(x, 'b'))

# print the resulting dataframe
print(f"After Encoding the Data:\n\n{df}\n")
```

Count Encoding:

• Count Encoding is a method for encoding categorical variables by **counting the number of times a category appears** in the dataset.

- For example, if a variable has categories ‘A’, ‘B’ and ‘C’ and category ‘A’ appears 10 times in the dataset, it will be assigned a value of 10.




```
# Count Encoding:
# create a sample dataframe with a categorical variable
df = pd.DataFrame({'fruit': ['apple', 'banana', 'apple', 'banana']})
print(f"Before Encoding the Data:\n\n{df}\n")

# perform count encoding on the 'fruit' column
counts = df['fruit'].value_counts()
df['fruit'] = df['fruit'].map(counts)

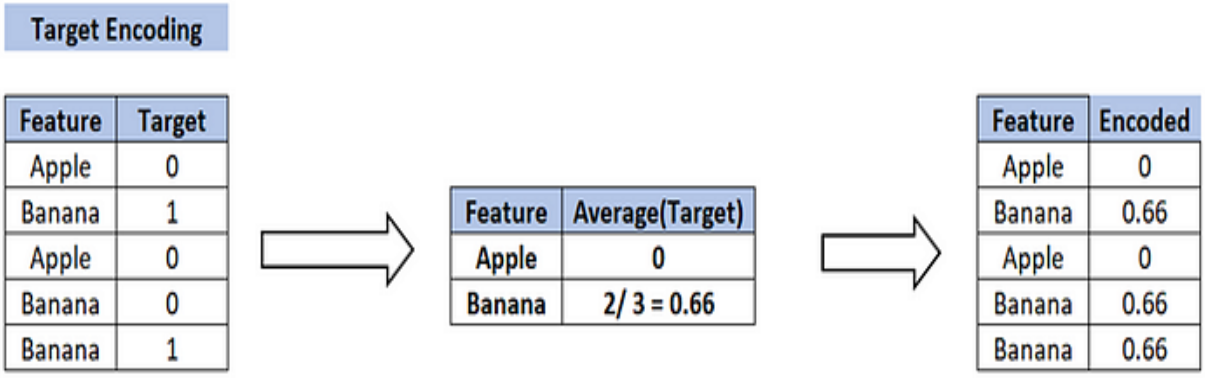
# print the resulting dataframe
print(f"After Encoding the Data:\n\n{df}\n")
```

Target Encoding:

• This is a more **advanced encoding technique** used for dealing with **high cardinality categorical features**, i.e., features with many unique categories.

• The average target value for each category is calculated and this average value is used to replace the categorical feature.

- This has the **advantage of considering the relationship between the target and the categorical feature**, but it can also **lead to overfitting** if not used with caution.




```
# Create a sample dataframe with categorical data and target
df = pd.DataFrame({'color': ['red', 'green', 'blue', 'red', 'green'],
                   'target': [0, 1, 0, 1, 0]})
print(f"Before Encoding the Data:\n\n{df}\n")

# Calculate the mean target value for each category
target_mean = df.groupby('color')['target'].mean()

# Replace the categorical data with the mean target value
df['color_label'] = df['color'].map(target_mean)


print(f"After Encoding the Data:\n\n{df}")
```

Categorical Variables: One-Hot Encoding, Label Encoding.

One Hot Encoding

One hot encoding is a technique that we use to represent categorical variables as numerical values in a machine learning model.

The advantages of using one hot encoding include:

1. It allows the use of categorical variables in models that require numerical input.
2. It can improve model performance by providing more information to the model about the categorical variable.
3. It can help to avoid the problem of ordinality, which can occur when a categorical variable has a natural ordering (e.g. “small”, “medium”, “large”).

The disadvantages of using one hot encoding include:

1. It can lead to increased dimensionality, as a separate column is created for each category in the variable. This can make the model more complex and slow to train.
2. It can lead to sparse data, as most observations will have a value of 0 in most of the one-hot encoded columns.
3. It can lead to overfitting, especially if there are many categories in the variable and the sample size is relatively small.
4. One-hot-encoding is a powerful technique to treat categorical data, but it can lead to increased dimensionality, sparsity, and overfitting. It is important to use it cautiously and consider other methods such as ordinal encoding or binary encoding.

One Hot Encoding Examples

In **One Hot Encoding**, the categorical parameters will prepare separate columns for both Male and Female labels. So, wherever there is a Male, the value will be 1 in the Male

column and 0 in the Female column, and vice-versa. Let’s understand with an example: Consider the data where fruits, their corresponding categorical values, and prices are given.

Fruit	Categorical value of fruit	Price
apple	1	5
mango	2	10
apple	1	15
orange	3	20

The output after applying one-hot encoding on the data is given as follows,

apple	mango	orange	price
1	0	0	5
0	1	0	10
1	0	0	15
0	0	1	20

Label Encoding

Label Encoding is a technique that is used to convert categorical columns into numerical ones so that they can be fitted by machine learning models which only take numerical data. It is an important pre-processing step in a machine-learning project.

Example Of Label Encoding

Suppose we have a column Height in some dataset that has elements as Tall, Medium, and short. To convert this categorical column into a numerical column we will apply label encoding to this column. After applying label encoding, the Height column is converted into a numerical column having elements 0,1, and 2 where 0 is the label for tall, 1 is the label for medium, and 2 is the label for short height.

Height	Height
Tall	0
Medium	1
Short	2